ABRA — methodology, and why each choice is the defensible one

Written because "we should test builds by simulating games" is not a method until you say how many games, against whom, varying what, and what result would count as real. Every section states the choice, the literature it rests on, and — where we have one — the measurement from this project that confirms or contradicts it. Negative results are kept in.

Plain-English summary first in each section; the arithmetic follows for anyone who wants it.

---

1. Why we run paired games with matched seeds

In plain terms. Every build is tested against the same opponents, in the same order, with the same dice. So when one build wins more, it is not because it drew easier matchups or luckier crits — it faced exactly what its rivals faced.

This is the standard variance-reduction technique in simulation, **common random numbers** (CRN): use the same pseudo-random numbers in the same ways across the competing configurations, so that observed differences are attributable to the configuration rather than to sampling noise. It is described as probably the most commonly used variance reduction technique, for simplicity and intuitive appeal (Goldsman, Georgia Tech ISyE 6644; KSL simulation text §9.2). Its use specifically inside multiple-comparison procedures — which is exactly our situation, many builds compared at once — is established in Nelson & Matejcik, *Management Science* 41(12) and Yang & Nelson, *Operations Research* 39(4).

The catch, and it is not decorative. CRN reduces variance only when outcomes across arms are positively correlated; the variance of a difference is `Var(A) + Var(B) - 2Cov(A,B)`, so if the pairing induces *negative* covariance it makes things worse. That failure mode is real and recently re-demonstrated (arXiv 2512.24145). `build_lab` therefore keeps the per-opponent win vectors and computes the paired difference directly, rather than subtracting two independently-estimated win rates — the pairing has to be *used*, not merely set up.

What it buys here. Unpaired, distinguishing a 3-point difference near 50% needs roughly 2,100 games per arm. Paired, with the matchup difficulty cancelling, the same resolution comes from a few hundred to ~2,000 depending on how correlated the arms are. The 2,000-per-cell figure used throughout is the conservative end and is not assumed to be enough — `build_lab` prints the smallest difference its actual sample can resolve, so an underpowered run says so.

---

2. Why full factorial, not one variable at a time

In plain terms. Change the item, then separately change a move, and you will never find out that Life Orb only pays off *when paired with* a particular move. You have to cross them.

This is the classic argument against one-factor-at-a-time. OFAT cannot detect interactions between factors, and factorial designs both reveal those interactions and need *fewer* runs for the same power — for three two-level factors, 8 runs against OFAT's 16, with the relative-efficiency advantage growing with every factor added (Czitrom, *The American Statistician*, "One-Factor-at-a-Time Versus Designed

Experiments"; Stat-Ease, *DOE Simplified* ch. 3). The reason factorials are cheaper is worth stating plainly: in a crossed design *every* game contributes to *every* factor's main effect at once, whereas OFAT throws away that shared information.

Confirmed here, on the first run. A 240-battle smoke test on Garchomp — small, and reported as small — already shows the shape: Adamant beats Jolly by about 10 points **under Life Orb**, and makes no difference at all **under Choice Scarf**. Neither an item sweep nor a spread sweep alone would have produced that sentence. This is the whole case for the design, and it appeared immediately.

Cost. Crossing move-combinations x items x spreads for the 14 most-played species is 1,050 cells, ~2.1M games, ~12.7 hours at the measured 46 games/sec. Affordable, and derived by `engine/set_space.js` rather than estimated.

---

3. Why the build space is derived, not chosen

In plain terms. We used to decide by hand that a move on 85%+ of sets was "locked". That number was invented. It is now computed from a fact about Pokémon: every set has exactly four moves.

Because Smogon's move percentages are shares of sets, and every set holds four moves, the listed percentages plus the "Other" bucket sum to 400 for every species. That identity does the work:

```
freedom = (400 - sum of the four most common moves) / 100
```

which reads as *how many of the four slots a real player changes away from the standard build.* Garchomp 0.71. Farigiraf 1.47. Kingambit 0.59. Across 259 species with 2,000+ teams the four most common moves account for **68% of every move slot played in the format**. Nothing here is a threshold.

Where a cutoff genuinely is needed — force a move onto every generated set, or roll for it — there is an exact answer requiring no judgement. If a move sits on a true fraction `p` of sets, always including it matches reality on `p` of sets, while sampling it with probability `p` matches on `p² + (1-p)²`. The difference is `(2p-1)(1-p)`, so **always-include wins exactly when p > 1/2**. Above half, force it; below half, roll. Kingambit's Sucker Punch at 99.4% is not a decision anyone makes.

Why we do not apply that strictly. Doing so leaves 95 of 259 species with all four slots forced and exactly one legal set, which contradicts the 35 distinct Garchomps in our own ladder store. So enumeration works by *substitution* into the standard four — drop one merely-usual move, add one alternate — which keeps every arm a set somebody actually runs while still differing from the reference in exactly one thing. Attributability is the point of the design; unrealistic arms waste games answering questions nobody asked.

---

4. The blind spot, stated up front

In plain terms. Smogon lists moves down to about 1% and dumps everything rarer into "Other". That bucket is 15–20% for nearly every Pokémon. Because a set is four moves, that works out to about **one real set in six containing a move we can never propose.**

```
P(set contains an unlisted move) = 1 - (1 - other/400)^4   ~= 17%
```

Median 17% across 283 species, worst 19%. **Kingambit is the exception at 3%** — its four best moves are Sucker Punch 99.4 / Kowtow Cleave 96.2 / Protect 76.0 / Iron Head 69.7 and there is genuinely almost nothing else. Species like that are solved; the rest are sampled.

This is a **floor on set realism**, not a bug to be fixed, and it is the reason not to keep chasing the last few points of diversity. It also sets the honest reading of any build-lab result: the comparison is over the visible space, and a sixth of the real space is not in it.

Spreads are worse and should be treated with more caution: Garchomp's Spreads "Other" is 38.4% against 19.8% for moves. Items are safe at 3.9%.

---

5. Why self-play alone cannot be the proof

In plain terms. A bot that only ever plays itself gets very good at beating itself. That is not the same as getting good at Pokémon.

This is the documented failure mode. Self-play overfits to the current opponent and forgets how to beat older versions, producing rock-paper-scissors strategy cycles rather than monotone improvement. The established remedies are fictitious self-play — train against a *mixture* of all previous versions rather than the latest — and league training, where **main agents** try to beat everyone while **exploiter agents** exist specifically to find and punish the main agent's weaknesses, and are then folded back into the opponent pool (AlphaStar, DeepMind; Minimax Exploiter, arXiv 2311.17190; opponent-aware league training, NeurIPS 2023). Opponents are sampled by how hard they are for the current learner, not uniformly.

What this dictates for ABRA. Two things are non-negotiable and both are still owed:

1. **A head-to-head gate.** Each new policy must beat the previous one above chance before it is adopted. Without it, an iteration loop can run indefinitely on a broken crank and every internal number will look fine.
2. **The transfer test.** Train on self-play, evaluate against *real ladder games*. It is the only check in this project that is not self-referential, and it has never been completed.

And it is why the end goal is the **real ladder**: it is external, adversarial, and the one measure self-play cannot fake.

Honest about the bar. VGC-Bench beat a professional on a *single fixed team* and degraded as team variety rose. The PokeAgent Challenge (NeurIPS 2025, 100+ entrants) concludes the general problem is unsolved, with large gaps between LLM agents, RL agents, and strong humans. "Stupidly high on the ladder" is a research target, not an engineering one. The fixed-team scoping in `docs/THEORY.md` §5 is the version reachable first.

---

6. Why results are corrected for multiple comparisons

Comparing 84 Garchomp builds at $p < 0.05$ produces about four "significant" results by chance alone even if every build is identical. `build_lab` applies **Benjamini–Hochberg** false-discovery-rate control across the whole family of tests in a run and reports the surviving cutoff. It also prints the count of tests, so a reader can see what the correction was applied across.

FDR rather than Bonferroni because these are exploratory screens over many arms where a few false positives are acceptable and being blind to real effects is not — the standard argument for FDR in

screening designs.

---

7. What every build-lab number is conditioned on, and why it is the open problem

Every win rate this project produces today is **a win rate in the hands of a weak pilot**: a policy that samples moves by usage frequency and does not read the board. Measured against real games, it lands super-effective moves 10.8% of the time against a human 23.4%, and plays moves that outright fail 8.7% against 2.7%.

So a build that scores well may simply be one that suits a bad player. That caveat is printed under every result table, and it is why **the scoring bot is the top item on the backlog**: it is what makes `build_lab` measure builds rather than measure the pilot.

Reproducing anything here

```
node engine/set_space.js --top 14        # the build space, freedom, and blind spot per species
node engine/realism_report.js            # generated vs real games, ranked by gap
node engine/build_lab.js --species Garchomp --builds 84 --games 2000
```

All three read Smogon data fetched monthly by `.github/workflows/smogon-stats.yml`. No month, cutoff, species list, or threshold is hardcoded anywhere in that path (S12/S13).